CHAITANYA
(Deemed to be University)
(Approved u/s. 3 of UGC act. 1956 by MHRD, Govt. of India)

Mini Project : Review 1

Repo Lens:

An AI-Powered GitHub Repository Analysis and Developer Profiling System

Goutham Vaishnav(23CSE3023)

Pernati Renee(23CSE3021)

Chilukuri Manoj (23CSE3002)

Mohammed Haji (23CSE3042)

Balineni Bhavani (23CSE3028)

Computer Science & Engineering

CSE A2- 6th Sem | 3rd Year

2023 - 2027 Batch

Guide Name : **Mr. Mahesh babu ,Assistant Professor ,Department of Computer science Engineering**

ABSTRACT

In the modern software industry, a developer's GitHub repository serves as a primary indicator of technical skills, coding standards, and project experience. It often functions as a professional portfolio for recruiters and hiring managers. However, despite its significance, many students, self-taught developers, and early-career professionals face challenges in understanding how repositories are evaluated, lack structured feedback on code quality and project organization, and are uncertain about effective ways to improve their projects.RepoLens is an AI-powered system designed to address these challenges by providing intelligent evaluations of public GitHub repositories. The system performs a comprehensive analysis across multiple dimensions, including code quality, project structure, documentation, testing practices, version control habits, and real-world applicability. By leveraging a combination of rule-based logic and large language models (LLMs), RepoLens generates:

1.A quantitative score reflecting repository quality

2.A human-readable evaluation summary

3.A personalized improvement roadmap

By delivering actionable feedback, RepoLens functions as a virtual coding mentor, helping developers identify strengths and weaknesses, enhance project quality, and align their work with industry expectations. The system facilitates continuous skill development and professional growth, enabling developers to transform their GitHub repositories into career-ready portfolios.

PROBLEM DEFINITION

GitHub repositories serve as a professional portfolio for developers, showcasing their skills, coding practices, and project experience.However, students, beginners, and early-career developers face several challenges:

1.Lack of understanding: They do not know how recruiters evaluate repositories.

2.No structured feedback: There is limited guidance on code quality, project organization, and documentation.

3.Unclear improvement roadmap: Developers lack personalized guidance to enhance their projects effectively.

Existing approaches have limitations:

1.Manual code reviews

2.Time-consuming

3.Inconsistent

4.Not scalable

5.Current tools focus mainly on:

6.Syntax checking

7.Linting

8.Limited code-level analysis

Problem Statement:

Currently, no intelligent system exists that can perform end-to-end evaluation of GitHub repositories, provide personalized, actionable feedback, and align recommendations with industry standards.

EXISTING SYSTEM & NEED FOR ANEW

Existing Systems:

Several tools exist for code analysis and GitHub repository evaluation:

1.Static Code Analysis Tools – SonarQube, ESLint, PylintEvaluate code quality metrics such as syntax, modularity, and cyclomatic complexity.

Limitation: Do not assess overall project structure, documentation, testing, or Git workflow.

2.Repository Analytics Tools – GitHub Insights, OpenHubProvide activity metrics including commits, pull requests, and contributors.

Limitation: Cannot measure developer skill or give actionable improvement guidance.

3.AI Coding Assistants – GitHub Copilot, Amazon CodeGuru Assist with code generation and optimization.

Limitation: Do not evaluate repository documentation, project organization, or version control habits.

Need for a New System:

No existing tool provides end-to-end evaluation of GitHub repositories. Current solutions lack the ability to provide personalized, mentor-like feedback. Developers require structured insights, skill assessment, and improvement roadmap to align projects with industry standards.

Solution:

RepoLens fills this gap by combining repository analysis, AI evaluation, and personalized recommendations to guide developers in improving their repositories effectively.

VIABLE OBJECTIVES

RepoLens is designed with clear, achievable, and measurable objectives to ensure project goals can be evaluated effectively:

1. Learning Effectiveness : Enhance developers' understanding of coding best practices, project structure, and documentation.

2. Engagement Rate : Encourage active participation in evaluating and improving repositories based on AI feedback.

3. Consistency in Performance : Provide reliable and repeatable evaluations across multiple repositories.

4. Personalized Strategy Accuracy: Generate tailored improvement roadmaps aligned with repository gaps and developer skill level.

5. Time-to-Learn Reduction : Minimize the time required to identify issues and implement improvements.

6. Technical Terminology / Jargon : Repository Metadata: Information about a repository such as contributors, languages, commits, and description.

- LLM (Large Language Model) : AI model used to generate human-like textual summaries and recommendations.

- Code Quality Metrics : Measures including cyclomatic complexity, modularity, and naming conventions.

- Version Control Practices : Frequency, quality of commits, branch usage, and pull request practices.

OBJECTIVES

The primary objectives of RepoLens are to provide developers with actionable insights, improve project quality, and support continuous skill development. The system aims to achieve the following:

1. Enhance Learning Effectiveness : Help developers understand coding best practices, project organization, and documentation standards.

2. Increase Engagement Rate : Encourage developers to actively analyze and improve their repositories based on AI-driven feedback.

3. Ensure Consistency in Performance : Provide reliable, structured evaluations across multiple repositories to maintain continuous improvement.

4. Improve Personalized Strategy Accuracy : Generate tailored improvement roadmap specific to each repository and developer skill level.

5. Reduce Time-to-Learn : Minimize the time required to identify issues and implement improvements by providing clear, actionable guidance.

6. Bridge the Gap Between Academia and Industry : Align student and beginner projects with industry standards, preparing developers for real-world expectations.

PROPOSED SYSTEM

RepoLens is an AI-powered GitHub repository evaluation and developer profiling system designed to address the limitations of existing tools. The system provides end-to-end assessment of public GitHub repositories and offers actionable insights to help developers improve their projects.

Purpose of the System

- To analyze GitHub repositories across multiple dimensions including code quality, project structure, documentation, testing practices, and Git version control habits.

- To generate a quantitative score, human-readable evaluation summary, and personalized improvement roadmap for each repository.

- To function as a virtual coding mentor, enabling continuous learning and professional development.

How It Overcomes Existing Problems:

1. Comprehensive Evaluation: Unlike static analyzers or analytics tools, RepoLens evaluates the repository as a whole, including structure, documentation, and real-world applicability.

2. AI-Driven Insights: Combines rule-based logic and large language models (LLMs) to generate human-like feedback and recommendations.

3. Personalized Guidance: Provides tailored suggestions based on repository gaps, skill level, and industry standards.

4. Scalable and Consistent: Capable of evaluating multiple repositories efficiently and consistently, eliminating manual review bottlenecks.

Key Features:

1. Repository Data Analysis: Fetches repository metadata, folder structure, commits, programming languages, documentation, and test coverage.

2. Multi-Dimensional Evaluation: Scores repositories on code quality, project structure, documentation clarity, testing practices, Git habits, and real-world relevance.

3. AI-Generated Summary: Produces mentor-like, human-readable feedback highlighting strengths, weaknesses, and areas for improvement.

4. Personalized Improvement Roadmap: Suggests short-term, mid-term, and long-term actions to enhance repository quality.

5. Skill Level & Scoring: Assigns a quantitative score (0–100) and skill level (Beginner / Intermediate / Advanced), along with badges (Bronze / Silver / Gold).

In essence, RepoLens is more than a code evaluation tool—it is a structured, AI-driven system that guides developers in building career-ready GitHub portfolios.

SYSTEM ARCHITECTURE

The RepoLens system architecture is designed to provide a comprehensive, AI-driven evaluation of GitHub repositories. It consists of five major layers, each performing a critical role in analyzing repositories and generating actionable insights.

1. Input Layer:

The user provides a public GitHub repository URL.

Acts as the entry point for the evaluation process.

2. Data Collection Layer:

Fetches repository data using the GitHub REST API.

Collected data includes:

-Repository metadata (tags, description)

-File and folder structure (folder tree)

-Commit history

-README and other documentation

-Programming languages used

3. Analysis Engine:

Performs multi-dimensional evaluation of the repository:

- Code Quality Analysis – LOC, complexity, naming conventions, linting checks

- Project Structure Evaluation – folder organization, architecture patterns

- Documentation Assessment – completeness and clarity of README, usage instructions, tech stack

- Testing & Maintainability Checks – test directories, unit/integration tests, maintainability metrics

- Version Control Analysis – commit frequency, branch usage, meaningful commit messages

4. AI Evaluation Layer

Uses Large Language Models (LLMs) to generate human-readable insights.

Key outputs:

- Evaluation Summary – strengths, weaknesses, and improvement suggestions

- Skill Assessment – Beginner, Intermediate, or Advanced

- Personalized Improvement Roadmap – step-by-step actionable guidance

5. Output Layer

Produces the final evaluation results:

- Score (0–100)

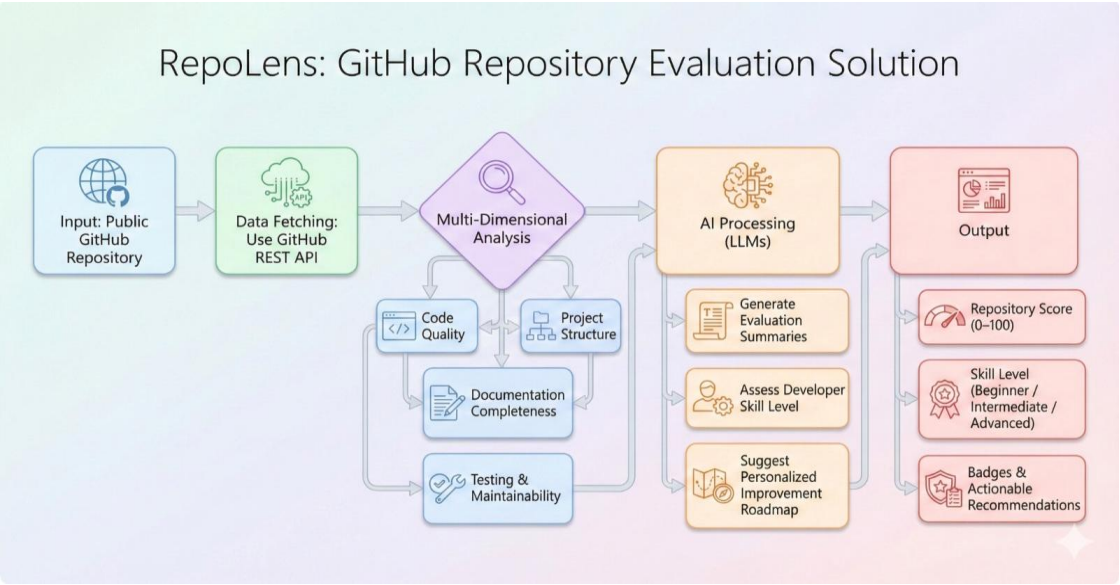
- Skill Level and Badge (Bronze, Silver, Gold)

- Improvement Recommendations for enhancing repository quality

Workflow Summary:

Input Layer → Data Collection → Analysis Engine → AI Evaluation → Output Layer

This architecture ensures a structured, scalable, and AI-driven evaluation process, bridging the gap between student projects and industry expectations.



MODULE-WISE DESCRIPTION

Overview:

The RepoLens system is composed of multiple modules, each responsible for specific tasks in the pipeline. Together, they ensure end-to-end evaluation of GitHub repositories and provide AI-driven feedback. Each module has a clear role and measurable functionality.

1. User Interface Module

Functionality:

-Provides a platform for users to submit public GitHub repository URL.

-Displays evaluation results, including scores, skill levels, and improvement recommendations.

-Ensures a user-friendly experience for both beginners and advanced developers.

2. Preprocessing Module

Functionality:

- Cleans and organizes repository data fetched from GitHub.

- Extracts files, commit history, README, and programming languages.

- Prepares data for code analysis, structure evaluation, and AI processing.

3. Machine Learning Module

Functionality:

- Uses LLMs and AI models to generate human-readable summaries.

- Identifies strengths and weaknesses in code, project structure, documentation, and Git practices.

- Produces personalized improvement roadmap for each repository.

4. Cognitive Techniques Repository Module

Functionality:

- Maintains evaluation rules, heuristics, and best-practice templates.

- Supports the AI evaluation module by providing structured knowledge for scoring and recommendations.

- Ensures consistency in feedback across repositories.

5. Personalization Engine Module

Functionality:

- Generates tailored improvement plans based on repository gaps and developer skill level.

- Assigns skill levels (Beginner / Intermediate / Advanced) and achievement badges (Bronze / Silver / Gold).

- Uses weighted scoring across evaluation dimensions to provide actionable and personalized guidance.

Module-Wise Functional Breakdown (Detailed Plan)

1. Repository Fetching Module

- Accepts GitHub repository URL.

- Fetches metadata using GitHub API.

- Extracts files, commits, and README/documentation.

2. Code Analysis Module

- Analyzes lines of code (LOC).

- Detects file size and modularity.

- Performs basic complexity checks.

3. Project Structure Module

- Analyzes folder organization.

- Detects architecture patterns (e.g., MVC, layered).

- Separates configuration files from source files.

4. Documentation Analysis Module

- Checks README presence.

- Evaluates section completeness (overview, installation, usage, tech stack).

- Measures clarity and usability.

5. Git Practices Module

- Analyzes commit frequency and consistency.

- Evaluates commit message quality.

- Checks branch usage.

6. AI Evaluation Module

- Generates evaluation summary using LLMs.

- Identifies strengths and weaknesses.

- Creates personalized improvement roadmap.

7. Scoring & Profiling Module

-Uses a weighted scoring system for all evaluation dimensions.

-Classifies skill level (Beginner / Intermediate / Advanced).

-Assigns achievement badges (Bronze / Silver / Gold).

TOOLS & TECHNOLOGY

The Repo Lens system leverages a combination of programming, AI, and web technologies to provide a scalable, user-friendly, and intelligent GitHub repository evaluation platform.

1. Programming Language

Python – Used for back-end development, data fetching, repository analysis, and AI/LLM integration.

JavaScript (React.js) – Used for frontend development to create a responsive and interactive user interface.

2. Data Storage

Since RepoLens fetches data directly from GitHub APIs, there is no permanent data storage required.

Temporary data handling is managed in-memory for preprocessing and analysis.

3. Frontend Technologies

React.js – Framework for building interactive web interfaces.

Tailwind CSS – For styling and responsive design, ensuring a clean and user-friendly experience.

HTML & CSS– Standard web technologies for structure and styling.

4. Development Environment

IDE: Visual Studio Code

Package Management: npm for frontend, pip for Python backend dependencies

Version Control: Git for source code management and collaboration

API Integration: GitHub REST API for repository data fetching

5. AI & Machine Learning Tools

Large Language Models (LLMs) – For generating human-readable evaluation summaries and personalized improvement suggestions.

Python Libraries: requests, flask, flask-cors, dotenv for backend processing and API communication.

Summary:

The combination of Python for back-end AI processing, React.js for frontend interactivity, and GitHub API integration ensures that RepoLens is both technically robust and user-friendly, capable of providing real-time, actionable insights for developers.

FEASIBILITY

The feasibility of the RepoLens system can be analyzed from three perspectives: technical, operational, and economic.

1. Technical Feasibility

RepoLens leverages existing technologies such as the GitHub REST API, Python, and AI/LLM models, which are mature and widely used.

The system architecture supports modular design, ensuring ease of integration for new evaluation metrics or AI models.

Frontend technologies like React.js and Tailwind CSS provide a responsive and user-friendly interface.

The AI evaluation module uses pre-trained large language models, minimizing the need for developing complex models from scratch.

2. Operational Feasibility

The system is user-friendly, requiring only a public GitHub repository URL for evaluation.

Evaluations are automatic and consistent, reducing the dependency on manual code reviews.

Provides actionable improvement roadmap, which can be easily followed by students, beginners, and early-career developers.

3. Economic Feasibility

Implementation requires minimal infrastructure, as data is fetched directly form GitHub and processed in-memory.

Cloud hosting or local servers can handle evaluation workloads efficiently, keeping operational costs low.

By automating repository evaluation, RepoLens reduces the need for manual mentorship or review, saving time and effort for educational institutions, recruiters, and developers.

CONCLUSION

RepoLens bridges the gap between student/early developer projects and industry expectations. By combining automated analysis, AI-driven insights, and personalized improvement guidance, the system enables developers to enhance repository quality, understand best practices, and build career-ready portfolios.